

Assignment #4: The Capstone Macroeconomic Engine

ECO 317 - Intermediate Macroeconomic Theory

Professor Jonathan Wolff

Miami University

Spring 2026

Format: Group Project (Max 6 students per group)

Tools: Python, Streamlit, Generative AI (ChatGPT/Gemini/Claude), Public Data APIs

1. Overview

Over the course of this semester, we have systematically built the toolkit of a modern macroeconomist. In our first assignment, we constructed an automated pipeline to confront theory with real-world data. In our second, we solved dynamic micro-founded household problems using Value Function Iteration. Most recently, we evaluated complex fiscal policy interventions within a medium-scale DSGE framework.

For your final project, you will integrate all of these modules into a single, unified "Macroeconomic Sandbox." In addition to combining your previous work into a seamless, tabbed web application, you will also build one final module: a simulation of long-run economic growth using the Solow Growth Model which we are currently completing in class.

The goal of this application is to create a comprehensive toolkit that allows a user to explore the macroeconomy across all horizons—from long-run steady-state growth down to high-frequency, short-run fiscal shocks, all grounded in live empirical data.

As always, you are acting as economic software architects. Use Generative AI to write the heavy programmatic lifting (API calls, dynamic programming algorithms, transition matrices, and Streamlit UI code), while you focus on enforcing rigorous economic logic, ensuring the budget constraints balance, and validating the mathematical moments.

2. Project Requirements

Your Streamlit application must feature a highly professional, tabbed interface (or a clean sidebar navigation list) that allows the user to seamlessly switch between each of your distinct macroeconomic modules (make them feel unified).

Global UI & Academic Styling: The dashboard must maintain a visually polished aesthetic, featuring a dark/navy background, bright white charting boxes (or white text on dark charts) for contrast, and utilizing the Garamond font throughout. Ensure all UI text, including tabs and widget labels, remains highly readable.

Module I: The Empirical Data Suite (From Assignment 1)

This tab must serve as the empirical anchor of your dashboard.

- **Automated Retrieval:** Fetch live data from APIs like FRED, BLS, or YahooFinance.
- **Processing & Visualization:** Clean the data, handle missing values, and calculate transformations (e.g., Log-Differencing, Year-over-Year inflation). Present professional graphs with multiple vertical axes, recession shading, and trend lines.
- **Econometrics:** Include interactive widgets to run a basic course-relevant econometric model (e.g., flexible OLS, local projection, VAR, etc. for proposed simple relationships) on the live data, displaying standard regression statistics.

Module II: Long-Run Growth – The Solow Model (NEW)

This new tab will focus on the long-run trajectory of the economy. Your code must simulate the Solow Growth Model with technological progress.

- **Interactive Parameters:** Include sidebar sliders for the saving rate (s), depreciation rate (δ), population growth rate (n), and the rate of technological progress (g), as well as the capital share of income (α) for a Cobb-Douglas production function $Y_t = K_t^\alpha (A_t L_t)^{1-\alpha}$.
- **Transition Dynamics:** Plot the time series of capital per effective worker (k_t), output per effective worker (y_t), and consumption per effective worker (c_t) as the economy transitions from an initial state to the steady state.
- **The Solow Diagram:** Generate the classic static plot showing the actual investment curve $s f(k)$ and the break-even investment line $(n + g + \delta)k$, with the steady-state dynamically updating as users move the sliders.
- **The Golden Rule:** Include an automated calculation (and visual marker) of the "Golden Rule" saving rate that maximizes steady-state consumption.

Module III: Micro-Founded Dynamic Models (From Assignment 2)

This tab brings the user into infinite-horizon dynamic programming, focusing on household behavior.

- **The Models:** Provide a sub-navigation to toggle between the Stochastic Consumption-Savings Model (CES preferences with income shocks), the Stochastic Robinson Crusoe Economy (capital accumulation with TFP shocks), and the Endogenous Labor Supply model.

- **Simulation & Moments:** Use Value Function Iteration (VFI) to solve the models, simulate at least 100 periods, and display key summary statistics (means, variances, autocorrelations).
- **Forecasting:** Allow the user to input specific near-term shock paths (e.g., "High" then "Low" TFP) to dynamically forecast the optimal consumption and savings responses.

Module IV: DSGE & Fiscal Policy (From Assignment 3)

This tab represents the ultimate synthesis: a medium-scale Smets-Wouters style DSGE model.

- **Global Frictions:** Maintain continuous sliders for Rule-of-Thumb household share, habit formation (h), capital utilization cost convexity (ψ), price/wage stickiness (θ_p, θ_w), and debt-feedback aggressiveness (ϕ_b).
- **Sub-Tab A: Model Fit:** Simulate a 1,000-period business cycle and display the 10 key unconditional moments (volatilities and correlations), proving the model accurately reflects stylized empirical facts.
- **Sub-Tab B: Fiscal Exercises:** Allow users to simulate Government Consumption, Government Investment, Labor Tax, or Capital Tax shocks. Crucially, the user must select a specific debt stabilization instrument (Ricardian lump-sum, specific tax hikes, or spending cuts). Display Impulse Response Functions (IRFs) and calculate Impact and Cumulative Multipliers.

Global Requirement: AI "Intelligent" Summaries

Every single module (tab) must feature dynamic text blocks at the bottom of the page. Utilizing Python f-strings, these blocks must intelligently synthesize the live outputs (e.g., identifying when the Solow economy reaches its steady state, interpreting a shift in the Golden Rule, or explaining the fiscal drag horizon in the DSGE model).

3. AI Prompting Guide & Helpful Tips

Integrating multiple complex scripts into one Streamlit application requires excellent software architecture. Be highly specific when prompting your AI tools.

- **Key Phrases to Include in Prompts:**
 - *Architecture:* "Create a Streamlit application with a main sidebar navigation that conditionally renders four distinct python modules/pages to keep the codebase clean and avoid memory leaks."
 - *Solow Model:* "Write a Python function to simulate the transition dynamics of the Solow model with Harrod-neutral technological progress. Ensure the function

analytically solves for the steady-state capital per effective worker k^* to anchor the graphs."

- *Performance:* "Ensure all heavy computational functions, specifically data API calls and the VFI solvers, are wrapped in `@st.cache_data` so the dashboard remains responsive when UI sliders are adjusted."
- **Suggestions for Added Features:** For the Solow module, consider adding a button that simulates a "sudden destruction of capital" (like a natural disaster or war) to visually demonstrate how an economy rapidly catches back up to its steady state via temporarily elevated growth rates.

4. The "Open Source" & AI Approach

- **AI Usage:** You are explicitly encouraged to use AI (ChatGPT, Gemini, Claude) to architect this final application. However, you must be the master of the code. If the AI hallucinates a mathematical function, uses a deprecated FRED API endpoint, or misspecifies the break-even investment equation in the Solow model, you must catch it and correct it.
- **Documentation:** Your code should be heavily commented. Use AI to generate robust docstrings for every major function so that anyone reviewing your repository understands exactly how the economics are being translated into mathematics.

5. Deliverable: The Live Demo

There is no written report for this assignment. Your final grade is based entirely on a comprehensive live demonstration.

- **Scheduling:** Teams will sign up for an extended 20-minute appointment slot with me.
- **The Demo:** You will bring your laptop, open your terminal, and run `streamlit run master_app.py` from scratch in front of me.
 - Does the master dashboard load cleanly without cross-tab interference?
 - Does the live data pipeline still function perfectly?
 - Do the Solow transition dynamics logically respond to shifts in the saving or population growth rates?
 - Are the VFI and DSGE multiplier mathematics still sound when integrated into the larger app?
- **Q&A:** I will ask group members to explain the economic mechanics driving the dashboard. Be prepared to explain the exact transmission mechanism of any shock in any tab, and to walk me through how your team utilized AI to solve specific programmatic bottlenecks.